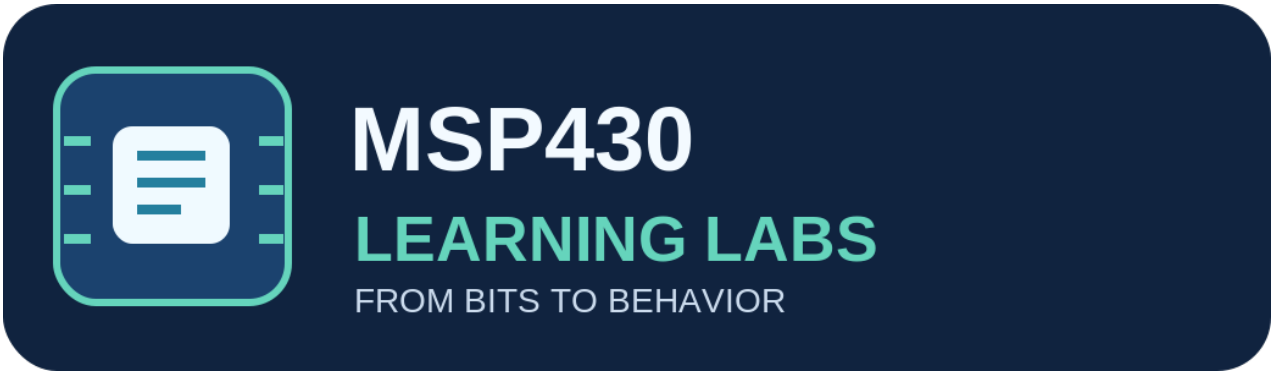


TEACHER'S GUIDE

From Bits to Behavior | Ten core lessons + four sensor investigations

A scaffolded microcontroller curriculum in which friendly vocabulary progressively reveals ordinary memory-mapped register C.

Target: MSP430G2553 • Boards: original MSP-EXP430G2 and newer MSP-EXP430G2ET

IDE: Texas Instruments Code Composer Studio 21.0.1 • Language: annotated C • UART: 9600 8N1

Validation: host syntax checked only; target compilation and classroom hardware qualification remain required.

The philosophy

Students begin with what the hardware does, then trace how C causes that action. The macro is a transparent scaffold, not a replacement for learning registers.

LED_ON(LED_A) → SET_BITS(P1OUT, BIT0) → P1OUT |= BIT0;

The same physical experiment can be repeated at all three levels. Students must predict behavior, manipulate registers, make measurements and explain their results in the language of both hardware and C.

Course at a glance

Lesson	Focus	Observable evidence
01 — Blink	Predict the LED waveform as high/low sections of equal duration.	Change 250 ms to 100 ms; compare predicted and observed cycle time.
02 — Button	Trace a pull-up resistor and a switch to ground.	Remove the 20 ms delay, evaluate bounce on ten slow presses.
03 — Patterns	Predict each of four mode transitions and two output phases.	Reverse the mode order and keep the same hardware interface.
04 — Timer	Calculate 1000 cycles/tick at 1 MHz and 500 ticks/half-cycle.	Change 500 ticks to 250 and measure LED timing.
05 — Interrupt	Predict changes when the ISR increments presses.	Explain what happens if P1IFG is never cleared; make timing diagram.
06 — PWM	Sketch high/low waveform for CCR1=250 out of CCR0=999.	Measure with DMM or logic analyzer if available; vary duty.
07 — ADC	Predict raw ADC for one-half of VCC and corresponding LED states.	Collect readings at five pot settings; plot reading vs position.
08 — UART transmit	Identify MCU UART TX, bridge chip, USB COM, terminal.	Collect a CSV file of ten samples at 9600 8N1.
09 — UART commands	Make a command table of A,B,O,?.	Add one new single-letter command and explain its effect.
10 — Telemetry capstone	Specify columns, sampling frequency, trial definition.	Export data to spreadsheet; compare ten readings at each setting.

Preparation and class routine

- Required per station: MSP430G2553 in G2/G2ET LaunchPad, USB cable, Windows computer with CCS 21 and terminal, 3.3V-compatible jumpers, breadboard, and DMM.
- Later labs: 10-kOhm pot, TMP36 analog temperature probe, HX711 with matched load cell, HC-SR04 with safe voltage divider, optional VL6180X breakout rated for 3.3V logic.
- Original G2 onboard debugger may not support current CCS; retain a supported external MSP430 programming option and check UART jumpers.
- Use one CCS project per lesson with main.c + edu430.h. Each project targets MSP430G2553; do not combine multiple main functions.
- Class routine: 10-minute prediction, 10-minute translation, 15-minute build, 15-minute observation, 20-minute investigation, 10-minute debrief.
- Student record: hypothesis, wiring/pin table, register explanation, experimental data or photo, one controlled change, conclusion and limitations.

Assessment by evidence

Prediction/inquiry (15), safety (15), working behavior (20), macro-to-register translation (20), calibration and units (15), clear reproducible report (15).

Classroom safety

The MSP430 inputs are 3.3 V logic. Connect no 5 V echo to a GPIO. Tie grounds together, confirm every external sensor pinout and test with a meter before powering circuits.

Lesson 01 — Blink

Driving question

What tells the chip that P1.0 is an output?

Before flashing

Predict the LED waveform as high/low sections of equal duration.

Inquiry extension

Change 250 ms to 100 ms; compare predicted and observed cycle time.

Register-level explanation

P1DIR selects output; P1OUT sets latch; |= only changes specified bits.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Model one complete register read/write on a whiteboard before introducing the macro.

Lesson 02 — Button

Driving question

Why does pressing a button read zero?

Before flashing

Trace a pull-up resistor and a switch to ground.

Inquiry extension

Remove the 20 ms delay, evaluate bounce on ten slow presses.

Register-level explanation

P1REN enables resistor; P1OUT chooses pull-up; P1IN samples logic.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Compare a pulled-high input with the button pressed to ground.

Lesson 03 — Patterns

Driving question

Is a button press an event or a level?

Before flashing

Predict each of four mode transitions and two output phases.

Inquiry extension

Reverse the mode order and keep the same hardware interface.

Register-level explanation

State is stored in a C variable, GPIO registers express that state.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Create a state transition graph with all four modes and release behavior.

Lesson 04 — Timer

Driving question

Why can a hardware timer be more reliable than counted delays?

Before flashing

Calculate 1000 cycles/tick at 1 MHz and 500 ticks/half-cycle.

Inquiry extension

Change 500 ticks to 250 and measure LED timing.

Register-level explanation

CCR0 sets period, CCIE enables interrupt, TA0CTL selects clock and mode.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Use a waveform drawing to distinguish busy delays from timer-driven ticks.

Lesson 05 — Interrupt

Driving question

What is the purpose of an interrupt flag?

Before flashing

Predict changes when the ISR increments presses.

Inquiry extension

Explain what happens if P1IFG is never cleared; make timing diagram.

Register-level explanation

IE enables; IES selects edge; IFG flags; ISR acknowledges event.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Have the student explain flag clear order and a mechanically bouncing switch.

Lesson 06 — PWM

Driving question

Is brightness caused by voltage or switching duty cycle?

Before flashing

Sketch high/low waveform for CCR1=250 out of CCR0=999.

Inquiry extension

Measure with DMM or logic analyzer if available; vary duty.

Register-level explanation

P1SEL routes a peripheral; CCTL1 sets PWM output mode.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Compare intensity and percentage duty cycle, not just high/low voltage.

Lesson 07 — ADC

Driving question

How are 10 bits related to a voltage?

Before flashing

Predict raw ADC for one-half of VCC and corresponding LED states.

Inquiry extension

Collect readings at five pot settings; plot reading vs position.

Register-level explanation

ADC10AE0 routes analog; ADC10MEM supplies sample counts.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Convert 0..1023 counts into a unit-bearing voltage using measured VCC.

Lesson 08 — UART transmit

Driving question

Why is UART not the same as USB?

Before flashing

Identify MCU UART TX, bridge chip, USB COM, terminal.

Inquiry extension

Collect a CSV file of ten samples at 9600 8N1.

Register-level explanation

USCI_A0 sends bytes; baud divisor matches configured clock.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Demonstrate the chain from MCU UART registers to USB bridge to Windows COM.

Lesson 09 — UART commands

Driving question

What changes when the PC sends input?

Before flashing

Make a command table of A,B,O,?.

Inquiry extension

Add one new single-letter command and explain its effect.

Register-level explanation

UCA0RXBUF exposes received byte; no proprietary protocol.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Use serial input to show that a remote control command changes local state.

Lesson 10 — Telemetry capstone

Driving question

When is a number a meaningful measurement?

Before flashing

Specify columns, sampling frequency, trial definition.

Inquiry extension

Export data to spreadsheet; compare ten readings at each setting.

Register-level explanation

Integrates state, ADC, reports; report assumptions and uncertainty.

Instructor checkpoint

Ask the student to expand a friendly macro, identify the exact MCU bit affected, state the predicted physical behavior, demonstrate it, and document one result.

Teaching note

Insist on CSV header, unit explanations and an accuracy limitation.

Four inquiry STEM appendices

A. TMP36 analog temperature

Read analog volts, calculate Celsius, compare with reference thermometer, analyze temperature offset, repeatability and VCC assumption.

B. HX711 strain gauge scale

Acquire signed 24-bit readings. Tare on an empty platform, calibrate counts per gram with known masses, evaluate drift, hysteresis and eccentric loading.

C. HC-SR04 ultrasonic time-of-flight

Measure echo pulse time using Timer1_A; convert with speed of sound. Sweep distance, plot residual error, and discuss room temperature and target reflectance. Use 5V-to-3.3V level divider.

D. ST VL6180X optical time-of-flight

Advanced experimental module: understand open-drain software I2C, 16-bit register addressing, manufacturer initialization, range/status reporting, and short optical measurement range. The breakout must be 3.3V logic compatible.

All four measurement appendices require bench validation before instructional use. Do not treat placeholder scale calibration as a real result.

Technical and original-source references

CCS: https://www.ti.com/tool/CCSTUDIO

G2553 datasheet: https://www.ti.com/lit/ds/symlink/msp430g2553.pdf

Original lab collection: https://github.com/ADolbyB/msp430g2553-labs

ST VL6180X setup: https://www.st.com/resource/en/design_tip/dm00123019-vl6180x-range-and-ambient-light-sensor-quick-setup-guide-stmicroelectronics.pdf

See docs/BOARD_COMPATIBILITY.md and docs/SENSOR_WIRING_AND_CALIBRATION.md for electrical instructions.

Distributed as GPL-3.0-or-later educational code, with source attribution. Independent work, not endorsed by TI or ST.

TEACHING THE EVOLUTION OF CODE

One four-mode LED problem, four increasingly readable solutions

Stage 1: Registers

```
P1OUT |= BIT0;
```

Introduce bit masks, direction, and physical GPIO.

Stage 2: Generic macros

```
SET_BITS(P1OUT, BIT0);
```

Explain how the preprocessor substitutes the same C operation.

Stage 3: Named actions

```
LED_ON(LED_A);
```

Connect observable intent to a particular output.

Stage 4: Functions

```
display(mode, phase);
```

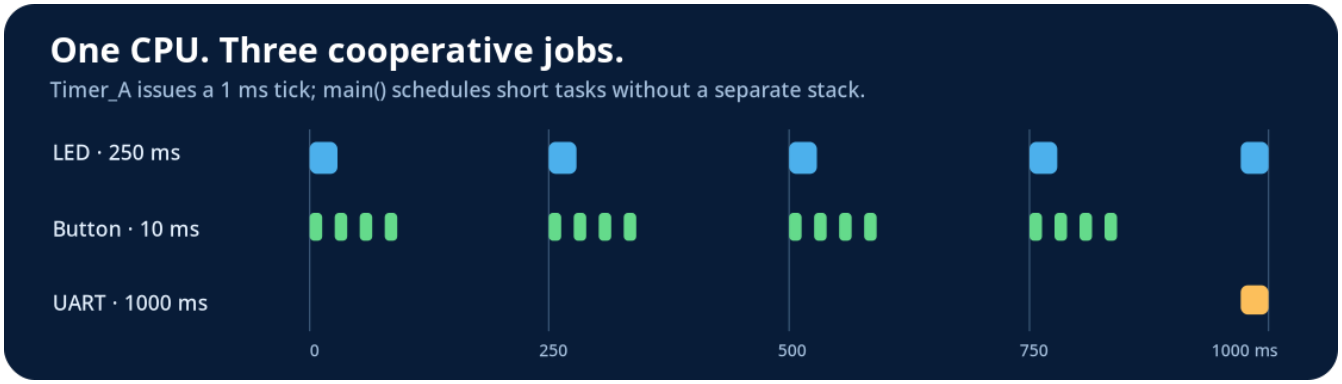
Separate state, hardware setup, and behavior.

Teacher demonstration: compare and predict

1. Make a truth table for the four modes before running any code.
2. Compile each stage in a separate MSP430G2553 CCS C project.
3. Compare compiled results, predict LED changes, and explain all register bits.
4. Discuss why timers, ADC, and protocols deserve functions, not complex macros.

A Tiny Multitasker You Can Understand

From blocking delays to register-level timing, readable scheduling macros, and a measurable low-power experiment.



One CPU executes three short jobs; Timer_A only maintains the millisecond clock.

Overview and hardware

Suggested time: Two class periods, 50–60 minutes each, plus optional EnergyTrace laboratory. Prerequisites: GPIO, switch inputs, Timer_A interrupts, and serial UART. Boards: MSP-EXP430G2 (original) or G2ET (new). EnergyTrace: newer G2ET only.

Stage	Application behavior	Concept revealed
01 · Blocking	LED heartbeat; button scanned only after long delays	Why short presses are lost
02 · Explicit timer	250 ms LED + 10 ms switch + 1000 ms UART	One ISR; three C jobs
03 · RUN_EVERY	Same tasks through a tiny macro/function	Abstraction reveals registers
04 · EnergyTrace	Switch between spin and LPM0 idle	Energy vs. execution style

Ask students

“How can one microcontroller keep an LED blinking, recognize button presses, and send a serial report without a real-time operating system?” Start with student predictions, then compare the actual data. An apparent multitasker is a sequence of short C functions scheduled by elapsed time.

Set up in CCS

Import each stage’s .projectspec file (or create an empty MSP430G2553 C project and copy just that stage’s source files). Stage 1/2 use main.c + edu430.h; Stage 3/4 add tiny_scheduler.h and button_filter.h. Match the LaunchPad programmer and target. For UART, use the LaunchPad Windows COM port at 9600 8N1. The ProjectSpec files need validation in a local CCS installation.

Investigation, Explanation, Transfer

Engage — run blocking Stage 1 (10 minutes)

Ask students to tap the onboard switch quickly during the LED delay. Compare ten intentional short taps to the reported count. What happened? The code checks the button only once each 250 ms busy wait. Button bounce also makes this a weak baseline.

Explore — run explicit Stage 2 (20 minutes)

Show Timer_A's 1 ms interrupt: a 1 MHz SMCLK, TA0CCR0 = 999, and a very short ISR that increments ticks_ms. Main tests 250, 10, and 1000 ms deadlines. Observe better responsiveness. Keep interrupts short; do not place UART printing inside the ISR.

Explain — expand Stage 3 (20 minutes)

Students should open tiny_scheduler.h and write out RUN_EVERY(led_job, now, blink_led). It expands to an if calling job_is_due(), whose unsigned elapsed-time check is (uint16_t)(now - last_run_ms). The blink_led() action still toggles P1OUT with BIT0. A macro makes the idea legible but does not alter the underlying scheduling mechanism.

Extend — deliberate overload (20 minutes)

Change only the LED interval from 250 ms to 100 ms. Observe behavior. Introduce a deliberately slow UART reporting job or temporary long delay; does button response worsen? Remove the delay afterward. Discuss what happens after time counter wrap, skipped deadlines, and long-running sensor transactions.

Evaluate — explain, not memorize (10 minutes)

Have teams submit a drawn timing sequence, an exact macro expansion, and before/after press observations. A good explanation distinguishes "the interrupt increments time" from "main executes jobs."

Data collection worksheet

Trial	Prediction	Observation / explanation
Stage 1: 10 fast taps	_____	_____
Stage 2: 10 fast taps	_____	_____
Stage 3: hold button	_____	_____
Stage 3: slow report task	_____	_____

Measure the Price of Busy-Waiting

On the newer MSP-EXP430G2ET, the built-in eZ-FET EnergyTrace circuit can compare the same tasks under two idle strategies. The G2553 supports EnergyTrace current/energy measurements, not EnergyTrace++ CPU-state tracing.

A controlled comparison

Independent variable: IDLE_LOW_POWER set to 0 (spin) or 1 (LPM0). Controls: same board, clock, calibration, task frequencies, wiring, debugger method, capture duration, and firmware options. Keep POWER_TEST_LED=0 and POWER_TEST_UART=0 in both builds. Dependent variables: average current (µA), energy (µJ or J), and/or average power (µW or W), as provided by CCS.

1. Compile Stage 4 with IDLE_LOW_POWER=0; capture a repeatable baseline.
2. Change only IDLE_LOW_POWER to 1; rebuild and capture the same duration.
3. Repeat both conditions, alternate their order, and note variation.
4. Compute percent current reduction from observed data—not an assumed number.
5. Optional: repeat with POWER_TEST_LED=1 in both builds to see the LED dominate energy.

Why LPM0?

In LPM0, the CPU stops but SMCLK continues. Timer_A uses SMCLK and still fires every 1 ms. The timer ISR must call __bic_SR_register_on_exit(LPM0_bits) so RETI resumes the cooperative loop. LPM3 would stop SMCLK: it is not interchangeable without changing the clock/timer design.

Condition	Mean current (µA)	Energy (µJ)	Observations
Active CPU	_____	_____	_____
LPM0 CPU	_____	_____	_____
Active repeat	_____	_____	_____
LPM0 repeat	_____	_____	_____

Assess (20 points)

4: prediction/data; 4: timer calculation and diagram; 4: accurate RUN_EVERY-to-register expansion; 4: cooperative scheduling and overrun explanation; 4: proposed improvement or controlled EnergyTrace analysis.

Teacher validation reminder

Host GCC syntax and scheduler tests passed, and ProjectSpec XML parses. These checks do not establish that Stage 4 has been compiled in TI CCS or measured on a G2ET. Record actual compiler versions, linker memory usage, debugger revision, measured current, and test date before reporting hardware-verified results.